

Code Explanation

Unit Test for Loading Module Explanation

Overview

The provided code is a unit test for the `load_to_target` function in the `loading` module. It uses Pytest and the `unittest.mock` library to mock the `SparkSession` and `DataFrame` objects, and to patch certain configurations.

The test case, `test_load_to_target`, verifies that the `load_to_target` function is working as expected by checking the calls made to the `write` method of the `DataFrame`.

Step 1: Importing necessary modules and defining fixtures

The test code starts by importing the necessary modules, including Pytest and `unittest.mock`. It then defines two fixtures: `mock_spark` and `mock_dataframe`, which create mock objects for `SparkSession` and `DataFrame` respectively.

```
import pytest
from unittest.mock import patch, MagicMock
from src.loading import load_to_target

@pytest.fixture
def mock_spark():
    return MagicMock()

@pytest.fixture
def mock_dataframe():
    df = MagicMock()
    writer = MagicMock()
    df.write = writer
    writer.format.return_value = writer
    writer.option.return_value = writer
    writer.mode.return_value = writer
    df.count.return_value = 100
    return df
```

Step 2: Defining the test case

The test case, `test_load_to_target`, takes the `mock_spark` and `mock_dataframe` fixtures as arguments.

```
def test_load_to_target(mock_spark, mock_dataframe):
```

Step 3: Patching configurations

The test case uses the `patch` function from `unittest.mock` to patch certain configurations, including `DB_CONFIG`, `TARGET_TABLE`, and `BATCH_SIZE`.

```
with patch('src.loading.DB_CONFIG', {
    "target": {
        "jdbc_url": "jdbc:test",
        "user": "user",
```

```

        "password": "password",
        "driver": "test.driver"
    },
    patch('src.loading.TARGET_TABLE', "TARGET_TABLE"),
    patch('src.loading.BATCH_SIZE', 1000):

```

Step 4: Calling the `load_to_target` function

The test case calls the `load_to_target` function with the `mock_spark` and `mock_dataframe` objects.

```
load_to_target(mock_spark, mock_dataframe)
```

Step 5: Verifying the calls made to the write method

The test case verifies that the `write` method of the `DataFrame` is called with the correct arguments.

```

mock_dataframe.write.format().assert_called_once_with("jdbc")
mock_dataframe.write.format().option().assert_any_call("url", "jdbc:te
st")
mock_dataframe.write.format().option().assert_any_call("dbtable", "TAR
GET_TABLE")
mock_dataframe.write.format().option().assert_any_call("user", "user")
mock_dataframe.write.format().option().assert_any_call("password", "pa
ssword")
mock_dataframe.write.format().option().assert_any_call("driver", "test
.driver")
mock_dataframe.write.format().option().assert_any_call("batchsize", 10
00)
mock_dataframe.write.format().option().option().option().option().op
tion().option().mode().assert_called_once_with("append")
mock_dataframe.write.format().option().option().option().option().op
tion().option().mode().save().assert_called_once()

```